LoanDesk — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where each lives

One job per file:

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py` — settings, named errors (`MissingConfigError`, two HTTP errors, `MaxIterationsExceeded` carrying the step log), one logging setup.
- `src/tools/calculator.py`, `fx.py`, `policy_search.py` — one tool each. The calculator is pure logic; `fx` calls a live API through `utils/http_client.py`; `policy_search` fails on its first call on purpose.
- `src/agents/worker.py` — the hand-built loop: `ALL_TOOLS`, `AgentResult`, `find_tool`, `run_tool` (returns `(result, error)`), `run_worker`.
- `src/agents/verifier.py` — `build_verifier_chain`, `collect_evidence` (successful steps only), `verify`.
- `src/services/finance_service.py` — `answer_task`: Worker → Verifier → one retry.
- `src/chains/direct_answer.py` — the no-tools baseline the compare script uses.

Build order that works: the direct chain (see it guess) → one tool + the loop → all three tools with the limit and recovery → the Verifier and the handoff.

Hint 2 — the three tools, and the flaky one

The calculator is exact amortization math — no network, perfect pure-logic tool:

```
monthly_rate = annual_rate_percent / 100 / 12
factor = (1 + monthly_rate) ** months
payment = principal * monthly_rate * factor / (factor - 1)   # guard rate==0
```

The FX tool hits the keyless **frankfurter.app** through Doc02's `request_with_retry` — a real live lookup, the way Project 2's weather tool used Open-Meteo. Return a string the model can read.

The policy search **raises on its first call**, then works:

```
class FlakyToolState:
    call_count = 0

@tool
def search_policies(query: str) -> str:
    FlakyToolState.call_count += 1
    if FlakyToolState.call_count == 1:
        raise ConnectionError("policy service timed out")
    ...
```

Write each tool's docstring to say **when to use it** — the model reads it. Say when *not* to use one, so the three don't get mixed up.

Hint 3 — the loop, and getting a failure back to the model

The loop is the standard tool-calling loop over LangChain messages:

```
model_with_tools = model.bind_tools(ALL_TOOLS)
messages = [SystemMessage(worker_prompt), HumanMessage(task)]
for _ in range(max_iterations):
    reply = model_with_tools.invoke(messages)
    messages.append(reply)              # append BEFORE the tool results
    if not reply.tool_calls:
        return AgentResult(reply.content, steps)
    for call in reply.tool_calls:                # call = {"name","args","id"}
        result, error = run_tool(call, ALL_TOOLS)
        observation = result if error is None else error
        messages.append(ToolMessage(content=observation,
                                    tool_call_id=call["id"]))
        steps.append({...})
raise MaxIterationsExceeded("...", steps)
```

Two things people get wrong:

- **Append the model's reply to `messages` before the tool results** — otherwise the `ToolMessage` has no `tool_call_id` to attach to, and the next call is a `400`.
- **`run_tool` catches the exception** (`ConnectionError`, `ValidationError`, anything) and returns it as a string, so the flaky tool's failure becomes an observation the model retries — not a crash.

Hint 4 — the Verifier, and the one retry

Make the verdict a real boolean, not text to grep:

```
class VerifierVerdict(BaseModel):
    approved: bool
    reason: str
```

Build it as `prompt | model.with_structured_output(VerifierVerdict)`, and feed it **only successful** tool results (`collect_evidence` skips any step whose result starts with `Error`). The service runs the Worker, verifies, and on rejection adds the reason to the task and runs the Worker **once** more, then verifies again. Test the Verifier with a deliberately wrong answer (tamper with `result.final_answer` after a real run) — if it still approves, it isn't really checking.

Still stuck? The whole thing, every file, is in the complete solution.